

EE446-TinyML-Lab5-Richard

Basic Information

- Name: Richard (Yunchao) He
- Student ID: 2301773
- Report document:
<https://docs.google.com/document/d/1tZDYlG0cHk4tSQxVUMMGhq4ziThaEeW57gmU6YHiOY/edit?tab=t.0>

Lab5 Questions and Answers

1. In the notebook, the audio waveform is transformed into time-frequency features before being passed to the keyword spotting model. What is the purpose of converting raw audio into spectrogram/MFCC-style features instead of directly using the raw time-domain waveform?
 - Raw time-domain audio contains complete information, but its sample-by-sample waveform is high-dimensional and can vary with the speaker, volume, timing, and recording conditions. It is difficult for a small model to interpret raw time-domain audio directly. Spectrogram/MFCC-style features represent how frequency content changes over time, exposing patterns that distinguish different spoken keywords. This provides a more compact and informative input, reducing model complexity and improving the efficiency and robustness of keyword spotting on TinyML devices.
2. During full integer quantization, the notebook uses a representative dataset. What is the purpose of the representative dataset, and why is it important for creating an INT8 TensorFlow Lite model?
 - The representative dataset provides a small set of typical model inputs for calibration during post-training quantization. TensorFlow Lite runs these samples through the FP32 model to estimate the numerical ranges of the input, output, and intermediate activations. It then uses these ranges to calculate the scale and zero-point needed to map floating-point values to INT8 values. The dataset is not used to retrain the model, but it should reflect the data expected during deployment. If it is not representative, the calibration ranges may be inaccurate, causing clipping or poor use of the

available INT8 range and reducing model accuracy. It is therefore essential for producing an accurate fully integer-quantized model suitable for integer-only inference on a microcontroller.

3. What does `audio_processor.get_data()` do? Refer to `input_data.py`.
 - `audio_processor.get_data()` is a python function in TensorFlow. It retrieves audio samples from the selected training, validation, or testing partition and converts them into the preprocessed features expected by the keyword spotting model. It selects the requested number of samples, applies time shifting, and may mix in background noise for training samples. Silence samples are generated by muting the foreground audio while retaining background noise. It then runs the TensorFlow preprocessing graph to produce a flattened time-frequency feature vector for each audio clip and converts each word label into its numeric class index. The function returns the processed feature data and the corresponding label indices. A `how_many` value of -1 requests the entire selected partition.
4. In the final model export step, the TensorFlow Lite model is converted into a C/C++ byte array. What are `g_model` and `g_model_len`, and why do they need to be copied correctly into the Arduino source file?
 - `g_model` is a C/C++ byte array containing the complete serialized TensorFlow Lite model, and `g_model_len` is the size of that array in bytes. Because an Arduino microcontroller generally cannot load a .tflite model from a normal file system at runtime, the model is embedded directly into the program and stored in flash memory. TensorFlow Lite Micro uses the array and its length to locate and interpret the model data. They must be copied correctly because missing, reordered, or modified bytes, or an incorrect length, can corrupt or truncate the FlatBuffer model and prevent the interpreter from initializing or performing inference correctly.
5. Please include a screenshot showing the correct identification of at least one keyword, either “YES” or “NO”, as displayed in the Arduino IDE Serial Monitor after deploying your KWS model on the Arduino Nano 33 BLE. The screenshot should be included in your submitted PDF.
 - The screenshot which can correctly identify “Yes”.
 - Note: my experiment could not identify “No”, which is a little bit weird. It can identify “Yes” only.

✓ → 🔍

🔌 Arduino Nano 33 BLE ▾

micro_speech_lab5.ino

arduino_audio_provider.cpp

arduino_command_responder.cpp

arduino_main.cpp

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

/* Copyright 2020 The TensorFlow Authors. All Rights Reserved.

Licensed under the Apache License, Version 2.0 (the "License");

you may not use this file except in compliance with the License.

You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software

distributed under the License is distributed on an "AS IS" BASIS,

WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.

See the License for the specific language governing permissions and

limitations under the License.

=====*/

#include <TensorFlowLite.h>

#include "main_functions.h"

#include "audio_provider.h"

Output

Serial Monitor ✕

Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem101')

Heard unknown (202) @78016ms

Heard unknown (204) @79984ms

Heard unknown (208) @85984ms

Heard unknown (201) @108784ms

Heard unknown (201) @125104ms

Heard unknown (206) @129328ms

Heard yes (203) @134624ms

Heard unknown (201) @136768ms

Heard yes (207) @140816ms

Heard unknown (201) @142704ms

Heard unknown (202) @166384ms

Heard unknown (201) @176880ms

Heard unknown (202) @180560ms

Heard unknown (201) @188624ms

Heard unknown (204) @192576ms

Heard unknown (201) @194464ms

Heard unknown (202) @200208ms

Heard unknown (207) @204784ms

Heard unknown (201) @212672ms

Heard unknown (202) @231248ms

Heard unknown (201) @233584ms

Heard unknown (203) @238960ms

Heard unknown (202) @243712ms

Heard unknown (206) @245952ms

Heard unknown (203) @250000ms

Heard yes (202) @265872ms

Heard unknown (205) @278432ms

Heard unknown (201) @294224ms

Heard unknown (201) @299440ms

Heard unknown (202) @301312ms

Heard unknown (203) @308048ms